

Using Machine Learning

Project Report

Dataset: AI4I 2020 Predictive Maintenance Dataset

Prepared from the current implemented project and verified model results

1. Abstract

This project implements an Intelligent Predictive Maintenance System using machine learning on the AI4I 2020 Predictive Maintenance Dataset. The system predicts whether a machine is likely to fail using machine type and five sensor-related inputs: air temperature, process temperature, rotational speed, torque, and tool wear. It also provides failure probability, a risk score and level, a likely failure type, local SHAP-based contributing factors, a maintenance recommendation, and an example cost-based business decision. The production binary failure model is XGBoost, selected using recall-focused F2 score because missing a real machine failure can be costly. On the current held-out evaluation set, XGBoost achieved recall of 0.847, F2 score of 0.774, and ROC-AUC of 0.972. The deployed decision threshold is 0.55. Cost figures shown by the application are demonstrative project assumptions and are not real company costs.

2. Problem Statement

Unexpected equipment failure can cause downtime, repair expense, safety concerns, and production loss. Traditional maintenance may be reactive or time-based, which can either miss failures or cause unnecessary maintenance. The problem addressed by this project is to use machine sensor readings to identify failure risk early and convert the machine learning result into an understandable maintenance decision.

3. Objectives

- Predict machine failure as a binary classification problem.
- Estimate the probability of failure and convert it into a 0-100 risk score.
- Identify a likely failure type using the dataset failure-mode labels.
- Explain local predictions using actual SHAP values from the selected model.
- Recommend an operational maintenance action based on risk and the model threshold.
- Estimate expected failure loss using clearly labelled demonstrative cost assumptions.
- Provide a simple Streamlit dashboard for project demonstration.

4. Dataset

The project uses the AI4I 2020 Predictive Maintenance Dataset stored in `Dataset/ai4i2020.csv`. The dataset contains 10,000 rows and 14 columns, as confirmed in the project data-understanding notebook. The primary target is Machine failure. The production input features are Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], and Tool wear [min]. The dataset also contains five failure-mode indicators: TWF, HDF, PWF, OSF, and RNF.

- Machine failure: binary target used by the main production model.
- Failure mode indicators: used to construct the auxiliary likely-failure-type model.
- Product ID and UDI: available in the dataset but not used as dashboard sensor inputs.

5. Data Cleaning

The reusable data-cleaning pipeline is implemented in `Src/data_loader.py`. The loader verifies that all required feature, target, and failure-indicator columns are present. The cleaning function removes duplicate rows, removes missing

values, and ensures that Machine failure is stored as an integer. The training pipeline writes the cleaned dataset to Output/cleaned_data.csv.

6. Exploratory Data Analysis (EDA)

EDA was performed during the learning and analysis stage in Notebooks/03_EDA.ipynb. The notebook contains multiple visual outputs. The project also includes an Images directory intended for chart exports; however, this report does not insert any chart that is not currently available as a valid file. The valid generated image available from the current training pipeline is Images/production_confusion_matrix.png.

7. Preprocessing

Preprocessing is centralized in Src/preprocessing.py to ensure that training and prediction use the same transformations. Machine Type is one-hot encoded with handling for unknown categories. Numeric sensor features are standardized using StandardScaler. The preprocessing object and classifier are stored together in a scikit-learn Pipeline, which prevents duplicate preprocessing logic and reduces the risk of training?prediction mismatch.

8. Models Compared

The current training script compares four binary classifiers. Class imbalance is addressed through class weighting for Logistic Regression, Decision Tree, and Random Forest, and through scale_pos_weight for XGBoost. The production decision is based on recall-focused F2, then recall and precision, rather than accuracy alone.

Model	Imbalance approach
Logistic Regression	class_weight = balanced
Decision Tree	class_weight = balanced
Random Forest	class_weight = balanced
XGBoost	scale_pos_weight based on training imbalance

9. Model Evaluation

The evaluation metrics are read from the current Output/evaluation_results.csv generated by the training pipeline. The system evaluates accuracy, precision, recall, F1, F2, ROC-AUC, and a selected decision threshold. Recall and F2 receive additional importance because false negatives can mean an actual failure was missed.

Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC	Threshold
XGBoost	0.9736	0.5760	0.8471	0.6857	0.7742	0.9723	0.55
Random Forest	0.9720	0.5620	0.8000	0.6602	0.7375	0.9703	0.45
Decision Tree	0.9716	0.5660	0.7059	0.6283	0.6726	0.8462	0.90
Logistic Regression	0.9136	0.2369	0.6941	0.3533	0.5008	0.8836	0.70

10. XGBoost Selection

XGBoost is the current production failure model. It was selected by the current training script because it achieved the highest F2 score among the compared models. Its verified metrics are Recall = 0.847, F2 = 0.774, and ROC-AUC = 0.972. This choice supports the project business requirement of reducing missed failure warnings while still providing

a probability score for decision-making.

11. Decision Threshold = 0.55

The stored production metadata sets the binary decision threshold to 0.55. The training code evaluates candidate thresholds from 0.05 to 0.95 in steps of 0.05 and selects the threshold that maximizes F2 score, with recall given more weight than precision. A machine is predicted as failure when its failure probability is greater than or equal to 0.55. This threshold is part of the current deployed model configuration and is not changed by the dashboard.

12. Failure-Type Prediction

Machine failure prediction and failure-type prediction are handled as separate tasks. The auxiliary failure-type model is trained only on failure rows with one unambiguous active failure-mode label. The project maps the dataset indicators to Tool Wear Failure, Heat Dissipation Failure, Power Failure, Overstrain Failure, and Random Failure. When the binary model predicts failure, the dashboard can display the likely failure type from this auxiliary model.

13. SHAP Explainability

Local explainability is implemented in `Src/explainability.py`. For supported tree-based models, the system transforms the entered input using the saved preprocessing pipeline and obtains SHAP values from the actual trained classifier. The dashboard lists the strongest local contributing features and indicates whether each feature increased or reduced the predicted failure risk. If an explanation is unavailable, the system intentionally displays no explanation rather than producing a fabricated one.

14. Risk Scoring

The application converts the predicted failure probability to a risk score from 0 to 100. The current project thresholds are: LOW below 25, MEDIUM from 25 to below 50, HIGH from 50 to below 75, and CRITICAL from 75 to 100. These are project decision thresholds used for dashboard interpretation.

15. Maintenance Decision

The maintenance decision is kept separate from the ML model output. If the failure probability reaches the selected decision threshold or the risk level is CRITICAL, the recommendation is Immediate Maintenance. Otherwise, the current logic maps HIGH to Prepare Maintenance, MEDIUM to Schedule Maintenance, and LOW to Continue Monitoring. This makes the final output easier for a non-technical user to interpret.

16. Cost Estimation

The dashboard estimates expected failure loss as failure probability multiplied by potential failure cost. The current values are demonstrative project assumptions only: potential failure cost = \$120,000 and preventive maintenance cost = \$8,000. The application compares expected failure loss with preventive maintenance cost to produce a business-decision message. These values must not be presented as real company costs.

17. Streamlit Dashboard

The Streamlit application is implemented in `App/app.py`. A user enters Machine Type, Air Temperature, Process Temperature, Rotational Speed, Torque, and Tool Wear. The dashboard then displays the machine-failure class, failure probability, risk score, risk level, likely failure type, local SHAP factors, maintenance recommendation, model threshold, example cost estimate, and business decision. Model artifacts are cached for efficient repeated use.

18. Results

- The current production model is XGBoost.
- Verified Recall: 0.847.
- Verified F2 score: 0.774.
- Verified ROC-AUC: 0.972.

- Current deployed failure decision threshold: 0.55.
- The current project generates cleaned_data.csv, evaluation_results.csv, and production_confusion_matrix.png during training.
- The dashboard uses saved joblib artifacts and does not retrain the model during prediction.

19. Conclusion

The Intelligent Predictive Maintenance System demonstrates an end-to-end machine learning workflow: validated data loading, reusable preprocessing, imbalance-aware model comparison, recall-focused threshold selection, saved production artifacts, prediction, failure-type estimation, SHAP explainability, risk scoring, maintenance recommendations, cost-based decision support, and a Streamlit interface. The selected XGBoost model achieved strong verified recall and ROC-AUC on the current evaluation output. The system is suitable as a college or internship project demonstration because it connects a technical prediction with a practical maintenance decision.

20. Future Enhancements

- Validate the system with additional real industrial datasets and time-based validation.
- Use organization-specific downtime, repair, and maintenance costs when available.
- Add monitored historical sensor trends and alert notifications.
- Evaluate probability calibration and business-cost-sensitive threshold tuning with stakeholder input.
- Add authenticated user access, prediction logging, and model monitoring for a production deployment.
- Include additional validated visualizations and dashboard screenshots in the final submission package.

Project Files Referenced

- Dataset/ai4i2020.csv
- Src/data_loader.py, Src/preprocessing.py, Src/train_model.py, Src/predict.py, Src/explainability.py, Src/utils.py
- App/app.py
- Models/failure_model.joblib, Models/failure_type_model.joblib, Models/model_metadata.joblib
- Output/evaluation_results.csv, Output/cleaned_data.csv
- Images/production_confusion_matrix.png